

Reviewer Pack — Krull Dimension Lower Bounds SOS Refutation Degree for 3-SAT

Entry e6b8ed98a856 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 08:45:51 UTC

Krull Dimension Lower Bounds SOS Refutation Degree for 3-SAT

Entry ID: e6b8ed98a856 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 08:45:51 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Krull Dimension of Ideals) **Field B** (complexity object): SOS Refutation Degree for 3-SAT Instances

Statement:

For a 3-SAT instance with m clauses over n variables, the SOS refutation degree required to prove unsatisfiability is at least the Krull dimension of the ideal generated by the clause polynomials (each clause is mapped to a polynomial via $(1 - x_i)(1 - x_j)(1 - x_k) = 0$ for a clause $(x_i \vee x_j \vee x_k)$).

Rationale (proposer’s reasoning):

The Krull dimension captures the ‘geometric complexity’ of the solution space’s algebraic variety. SOS refutations must account for this structure, suggesting a lower bound tied to the variety’s intrinsic dimension. This bridges algebraic geometry with proof complexity via polynomial optimization.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): db1368baaa1003b1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_3sat_instance(n: int, m: int) -> list:
    clauses = []
    variables = set(range(1, n + 1))
    for _ in range(m):
        clause = random.sample(variables, 3)
        clauses.append(clause)
    return clauses

def polynomial(x, y, z):
    return (1 - x) * (1 - y) * (1 - z)

def construct_ideal(clauses: list) -> dict:
    ideal = {}
    for clause in clauses:
        monomials = set()
        for i in range(2**len(clause)):
            factors = [1]
            for j, var in enumerate(clause):
                if i & (1 << j):
                    factors.append(-var)
                else:
                    factors.append(var - 1)
            monomial = math.prod(factors)
            monomials.add(monomial)
        ideal[tuple(sorted(monomials))] = 0
    return ideal

def grobner_basis(ideal: dict) -> set:
    basis = list(ideal.keys())
    while True:
        new_elements = []
        for i in range(len(basis)):
            for j in range(i + 1, len(basis)):
                gcd = set()
```

```

        for monom_i in basis[i]:
            for monom_j in basis[j]:
                if monom_i == monom_j:
                    gcd.add(monom_i)
                    break
            if gcd:
                new_elements.append(tuple(sorted(gcd)))
        if not new_elements:
            break
        basis.extend(new_elements)
    return set(basis)

def krull_dimension(basis: set) -> int:
    return len(basis)

def sos_refutation_degree(clauses: list, p=2) -> int:
    n = max(max(abs(x) for clause in clauses for x in clause), key=abs)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    b = [0] * (n + 1)
    for clause in clauses:
        for i in range(2**len(clause)):
            factors = [1]
            for j, var in enumerate(clause):
                if i & (1 << j):
                    factors.append(-var)
                else:
                    factors.append(var - 1)
            monomial = math.prod(factors)
            row = [0] * (n + 1)
            for factor in factors:
                if isinstance(factor, int):
                    row[abs(factor)] += 1
                else:
                    row[factor] -= 1
            A[monomial] = row
            b[monomial] += monomial ** p
    # Solve the linear system Ax = b using Gaussian elimination
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            return -1 # No solution
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n + 1):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    return sum(1 for row in A if any(x != 0 for x in row))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n, min(2 * n, 30))
    clauses = generate_3sat_instance(n, m)
    ideal = construct_ideal(clauses)
    basis = grobner_basis(ideal)
    dim_I = krull_dimension(basis)

```

```

refutation_degree = sos_refutation_degree(clauses)
conjecture_holds = refutation_degree >= dim_I
counterexample = "" if conjecture_holds else f"refutation_degree={refutation_degree}, dim(I)={dim_I}"
return {
    "metric_name": "SOS Refutation Degree",
    "metric_value": refutation_degree,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30 * 4 + 2, 2))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_097a0593.py", line 124, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_097a0593.py", line 103, in run_trial

```

    m = random.randint(n, min(2 * n, 30))
    ~~~~~

```

File "/usr/lib/python3.12/random.py", line 336, in randint

```

    return self.randrange(a, b+1)
    ~~~~~

```

File "/usr/lib/python3.12/random.py", line 319, in randrange

```

    raise ValueError(f"empty range in randrange({start}, {stop})")

```

ValueError: empty range in randrange(33, 31)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid randrange parameters (33, 31), preventing data collection. Support condition cannot be evaluated without valid results. | next: Fix the test parameters to ensure valid randrange ranges (stop >= start) and re-run with proper seed handling

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111081
2	propose	ollama_remote	qwen3:8b	0	0	74357
3	preregistration	ollama_remote	qwen3:8b	0	0	24197
4	novelty	ollama_remote	qwen3:8b	0	0	20796
5	novelty	ollama_remote	qwen3:8b	0	0	15788
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21977
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13693
8	judge	ollama_remote	qwen3:8b	0	0	17435

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 299323 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/e6b8ed98a856.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e6b8ed98a856.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e6b8ed98a856.tar.gz (if generated)